

# A Strongly Typed C-like Subset Lowered to AArch64

Typed checking, normalization, control flow, and explicit effects

16 August 2026

## Abstract

This note extends the compact Python-like compiler into a conventional strongly typed C-like subset. The supported core contains integers, booleans, strings, fixed-size integer arrays, function definitions with typed parameters, conditionals, explicit array indexing, and one documented Linux AArch64 system call. A static checker validates the indexed result type before normalization and lowering. The emitter uses a small deterministic register convention and stack spills for binary expressions; it does not introduce dynamic language features or hidden runtime semantics.

## 1 Language and types

Types are `int`, `bool`, `string`, fixed arrays `int[n]`, and `void`. Expressions include literals, variables, arrays, indexing, arithmetic, equality, boolean values, conditionals, and calls. A function has the type

$$(\tau_1, \dots, \tau_n) \rightarrow \tau.$$

The checker maintains a value environment and a function environment. It rejects an expression unless every operator, branch, array index, and call has the required type. In particular, both branches of an ‘if’ must have the same result type.

## 2 Normalization invariant

The normalizer folds static arithmetic and boolean expressions before code generation. For example, the condition ‘`2 == 2`’ becomes ‘`true`’, although the generated sample retains the conditional to demonstrate control-flow lowering. A typed transformation preserves the result type:

$$\Gamma \vdash e : \tau \implies \Gamma \vdash \text{normalize}(e) : \tau.$$

The emitter consumes only checked expressions, so normalization cannot turn an integer expression into a string or an array into a scalar.

## 3 ABI subset

The compiler documents a small AArch64 convention:

- integer and boolean values use general registers;
- the first three call arguments use  $x_0, x_1, x_2$ ;
- function results use  $x_0$ ;
- $x_{29}$  is the frame pointer and  $x_{30}$  the link register;
- binary-expression temporaries spill through 16-byte stack slots;

- ‘sys\_write(fd, buffer, length)’ uses Linux syscall number 64 in  $x_8$  and executes ‘svc #0’.

No implicit heap, exception system, closure conversion, or dynamic dispatch is added.

## 4 Sample program

The AST-built sample contains a two-argument integer function ‘add’, a fixed integer array and index, a string passed to ‘sys\_write’, a boolean equality condition, and an integer conditional return. The checker reports ‘typecheck: PASS’. The generated assembly contains ‘add’, ‘main’, ‘cset x0, eq’, conditional branches, ‘sys\_write’, and ‘.rodata’ entries for the array and string.

## 5 Implementation

The implementation is `typed_subset.cpp`:

```
g++ -std=c++23 -Wall -Wextra -pedantic -O2 typed_subset.cpp -o typed_subset
./typed_subset > typed_subset.s
```

The typed AST is explicit and immutable. The code generator uses a small register map for parameters and locals, with stack preservation around binary operations. This is a simple register-allocation policy, not a global graph-coloring allocator; its predictability is useful while the type and lowering invariants are being established.

## 6 Boundaries and extensions

The compiler intentionally excludes implicit coercions, objects, exceptions, garbage collection, variadic calls, and undocumented runtime helpers. The next principled additions are a parser for the already-defined AST, explicit effect types for I/O, ABI-checked function declarations, and a verified lowering relation. Each should preserve

$$\Gamma \vdash e : \tau \implies \text{normalize}(e) : \tau \implies \text{emit}(e) : \text{Asm}(\tau).$$